

Automated Dynamic CV Pipeline

Complete Technical Roadmap for Dynamic Portfolio Ingestion, Multi-CV Aggregation, Factual Integrity & ATS Optimization

Target Workspace: Jobs Automation / Workflow

Live Portfolio: portfolio.onl9.club

ATS Engine: ats.onl9.club API

Output: 2-Page CV + 1-Page CL

1. Executive Summary & Problem Diagnosis

Currently, the pipeline in `Workflow` relies on a static curated profile (`candidate_profile.json`) and has fixed employer names (*Neurix, Datacom, Department of Education, Mitre10 MEGA*) and project lists hardcoded directly into the LLM prompt strings and integrity validators in `job_application_pipeline.js` .

While `my_cvs/*.pdf` is parsed for character length, its extracted content is never injected into the LLM generation prompt. Furthermore, the live portfolio website (`portfolio.onl9.club`) is not fetched or scraped anywhere in the codebase. As a result, adding a new CV to `my_cvs/` or updating the portfolio website does not update the generated application documents.

Target Goal

Transform the pipeline into a **self-updating, dynamic career engine** that automatically discovers, extracts, and merges all changes from `portfolio.onl9.club` and any number of CV PDFs in `my_cvs/` into an active candidate profile, while strictly preserving 100% factual accuracy, ATS 90%+ match scoring, exact 2-page CV rendering (≥92% fill), 1-page cover letter rendering, and automated Notion synchronization.

2. System Architecture Comparison

Component	Current Behavior	Automated Dynamic Architecture
Portfolio Site	Completely unused; zero network calls to <code>portfolio.onl9.club</code> .	Dynamic Puppeteer scraper extracts work experience, skills, projects, and learning labs with local caching (24h TTL) and offline fallback.
my_cvs/ PDFs	Reads text into <code>merged_cvs_source.txt</code> only for character check; never passed to LLM.	Deep semantic parser extracts bullet points, skills, certifications, and updates manifest via SHA-256 diffing.
Profile Storage	Static <code>candidate_profile.json</code> edited manually.	Unified profile aggregator merges Portfolio + my_cvs + Base JSON into an active profile with deduplication.
LLM Prompts	Hardcoded 4 employers, 3 projects, and 2 schools in string literals.	Dynamic template builder formats verified facts, employers, and projects on-the-fly from the active profile.
Integrity Gate	Static assertion checking against fixed list; bug passing <code>mergedText</code> instead of profile on lines 2234/2251.	Dynamic validator verifies that all active profile entries are present, with fixed signature calls and zero false positives.
Document Limits	Strict 2 full A4 pages (CV) and 1 page (CL).	Maintained with 100% fidelity: dynamic bullet weighting, font-size tuning (7.5pt–8.2pt), and page checks.

3. Step-by-Step Implementation Plan (7 Phases, 21 Steps)

Phase 1: Live Portfolio Ingestion Engine (`portfolio_scraper.js`)

Step 1.1

Scraper Core & DOM Selector Architecture

Pending

Develop `portfolio_scraper.js` using Puppeteer and DOM extraction to parse `https://portfolio.onl9.club`.
Extract:

- **Header & Contact:** Name, Title, Location, LinkedIn, GitHub, Email, Phone.
- **Work Experience:** Company (e.g. Neurix, Mitre10, Datacom, Woolworths), Role title, Dates (start/end), Location, and Responsibilities.
- **Education:** Unitec (Master of Applied Tech), Maharaja Surajmal (BCA), dates, and degrees.
- **Skills & Technologies:** Extract all 8 categories (Infrastructure/Cloud, Automation/IaC, CI/CD, Containers, Monitoring, Security, AI, Other).
- **Projects & Learning Labs:** SadServers, KodeKloud, Iximiuz, Next.js Portfolio, Learning Physics, ONL9 Toolkit, Job Pipeline.

Step 1.2

Resilient Local Caching & Offline Fallback

Pending

Implement cache management (`portfolio_cache.json`) with 24-hour TTL. If the site is unreachable (offline, DNS failure, slow Wi-Fi), the scraper transparently falls back to the cache without throwing errors or interrupting the pipeline run.

Step 1.3

Data Normalization & Sanitization

Pending

Normalize date formats (e.g., "August 2025 – Present"), clean HTML artifacts, decode Cloudflare email obfuscation, and structure records into canonical JSON schema.

Phase 2: Multi-CV PDF Text & Section Parser (`cv_parser.js`)

Step 2.1

Dynamic PDF Discovery & SHA-256 Hashing

Pending

Scan `my_cvs/*.pdf` dynamically. Compute SHA-256 hashes for each file to detect when a CV has been added, updated, or removed, avoiding redundant parsing while catching every update instantly.

Step 2.2

Sectional & Semantic Entity Extractor

Pending

Use `pdf-parse` coupled with heuristic regex and structural tokenizers to extract distinct sections from any source CV: Professional Summary, Experience bullets, Skills, Education, Projects, and Certifications.

Step 2.3

Source Traceability & Granular Tagging

Pending

Tag each extracted achievement bullet with its originating source file (e.g. `cv_linux_devops.pdf` vs `cv_support_infrastructure.pdf`) for factual auditability.

Phase 3: Unified Profile Aggregator (`profile_aggregator.js`)

Step 3.1

Intelligent Merge & Conflict Resolution Engine

Pending

Combine data across 3 sources: Base `candidate_profile.json` + Live `portfolio.on19.club` + Extracted `my_cvs/*.pdf`.

- **Employer Alignment:** Map company aliases (e.g., "Datacom" = "Datacom NZ", "Neurix" = "Neurix Limited").
- **Bullet Deduplication:** Compute string similarity (Jaccard / Levenshtein) to retain the highest-fidelity, metric-rich version of duplicate accomplishments.
- **Skill Union:** Merge all skill sets, preserving categorizations and tags.

Step 3.2

Dynamic Keyword & Tag Indexing for ATS Classifier

Pending

Auto-generate normalized search tags for new skills and experience entries to seamlessly feed `classifyJob()` and `scoreForJob()` in the pipeline.

Step 3.3

Safe Profile Persistence & Backup

Pending

Save the aggregated profile to `candidate_profile.json` (with an automated timestamped backup in `candidate_profile.backup.json`) ensuring immediate visibility and full manual override support.

Phase 4: Decouple Hardcoded Strings in Pipeline Prompts

Step 4.1

Dynamic LLM Prompt Generation

Pending

Refactor `generateCVAndCoverLetter()` in `job_application_pipeline.js` :

- Replace hardcoded 4 employers with `candidateProfile.experience.map(...)` .
- Replace hardcoded projects with `candidateProfile.projects.map(...)` .
- Replace hardcoded education with `candidateProfile.education.map(...)` .
- Dynamically generate strict factual boundaries based on whatever entities exist in the active profile.

Step 4.2

Dynamic Baseline Document Formatter

Pending

Update `buildFactualApplicationDocuments()` to dynamically format any number of experience records, project items, and skill matrices without fixed-length assumptions.

Step 4.3

Dynamic Repair & Re-fit Prompts

Pending

Refactor the self-repair prompts (lines 2221-2226) so they dynamically pull required employers, schools, and projects from the profile rather than static strings.

Phase 5: Dynamic Integrity Validator (`validateCVIntegrity`)

Step 5.1

Dynamic Factual Entity Verification

Pending

Update `validateCVIntegrity(cvMarkdown, profile)` to verify presence of every employer, role title, project, and educational institution dynamically extracted from `profile` .

Step 5.2

Fix Repair Function Signature Bug

Pending

Fix critical bug in `job_application_pipeline.js` lines 2234 and 2251 where `mergedText` (raw string) was erroneously passed instead of `candidateProfile` (object).

Step 5.3

Hallucination & Section Ordering Rules

Pending

Maintain strict anti-fabrication rules, section order checks, placeholder detection, and word-budget validation (750 to 1200 words) tailored to the dynamic profile content.

Phase 6: Core Pipeline Orchestrator Integration & UI

Step 6.1

Preflight Sync Lifecycle Hook

Pending

Integrate profile sync into `JobApplicationPipeline.run()` before job scraping/document generation. Output clear preflight logs:

```
[Profile Sync] Live portfolio fetched (4 roles, 46 skills) | 2 source CVs parsed | Profile aggregated
```

Step 6.2

Environment Variables & Config Controls

Pending

Add environment controls to `.env` and `.env.example`:

- `PORTFOLIO_URL=https://portfolio.on19.club`
- `SYNC_PORTFOLIO_ON_RUN=true` (default: true)
- `PORTFOLIO_CACHE_HOURS=24`

Step 6.3

Web UI & CLI Information Display

Pending

Update `job_application_form.html` and `server.js` to display the count of active CVs in `my_cvs/`, the last portfolio sync timestamp, and status pills.

Phase 7: Comprehensive Verification & Zero-Error Validation

Step 7.1

Automated Unit & Smoke Tests

Pending

Create unit test suite `tests/dynamic_sync_test.js` covering:

- Scraper extraction against live `portfolio.on19.club` and mock offline fallback.
- Multi-CV parsing and entity extraction across varied PDF layouts.
- Profile aggregator deduplication and conflict resolution.
- Dynamic integrity validation under edge cases.

Step 7.2

Page Budget & Visual Layout Verification

Pending

Execute test pipeline runs ensuring that regardless of new experience or skills added:

- CV PDF fills **EXACTLY 2 full A4 pages** with content height $\geq 92\%$ of page 2.
- Cover Letter PDF fits **EXACTLY 1 full A4 page** without overflowing.

Step 7.3

End-to-End Pipeline, ATS & Notion Sync Validation

Pending

Run full end-to-end execution with a real job link. Confirm ATS score check via `ats.on19.club` (target 90%+), automatic Notion database upload with PDF attachments, and automatic cleanup of local workflow files in `output/`.

4. Execution Order & Milestones

Order	Phase / Module	Key Deliverable	Validation Check
Phase 1	Portfolio Ingestion	<code>portfolio_scraper.js</code> + <code>portfolio_cache.json</code>	Puppeteer parses live site + offline fallback test passes
Phase 2	CV PDF Extractor	<code>cv_parser.js</code> + PDF hashing manifest	Extracts bullets & skills from any PDF in <code>my_cvs/</code>
Phase 3	Profile Aggregator	<code>profile_aggregator.js</code> + sync logic	Deduplicates & updates <code>candidate_profile.json</code>
Phase 4	Prompt Decoupling	Dynamic prompt generator in <code>job_application_pipeline.js</code>	No hardcoded employer or project strings in prompts
Phase 5	Dynamic Integrity Gate	Refactored <code>validateCVIntegrity()</code> + bug fix	Validates dynamic entities + zero false errors
Phase 6	Pipeline Integration	Orchestrator preflight hook + <code>.env</code> + Web UI	Web form and CLI show active CV count & portfolio status
Phase 7	Zero-Error Verification	Full test suite + E2E test run	2-page CV ($\geq 92\%$), 1-page CL, ATS score 90%+, Notion sync ok

Strict Stability Guarantee

Every step will be implemented sequentially, tested in isolation, and verified end-to-end. At no point will the existing multi-provider fallback chain (OpenRouter → Groq → NIM), the ATS evaluation loop, or the Notion upload integration be disrupted.